Tema 1

Desarrollo del Software

Entornos de desarrollo

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos 4
1. Introducción 5
2. Concepto de programa informático 6
3. Ejecución de programas en ordenadores 9
3.1 Datos y programas 9
3.2 Hardware y software 9
3.3 Estructura funcional de un ordenador: procesador, memoria 10
3.4 Tipos de software. BIOS. Sistema. Aplicaciones 10
3.5 Código fuente, código objeto y código ejecutable, máquinas virtuales 11
3.5.1 Código fuente 12
3.5.2 Código objeto y código ejecutable 12
3.5.3 Bytecode y las máquinas virtuales 15
4. Lenguajes de programación 17
4.1 Tipos de lenguajes de programación 17
4.1.1 Por un nivel de abstracción y generación 17
4.1.2 Por paradigma 19
4.1.3 Por forma de ejecución 20
4.2 Características de los lenguajes difundidos 22
5. Introducción al Ingeniería del Software 25
5.1 Proceso del software y ciclo de vida del software 25
5.2 Fases del desarrollo de una aplicación: análisis, diseño, codificación, pruebas, documentación, explotación y mantenimiento, entre otras 26
5.2.1 Fases del ciclo de vida 26
5.2.2 Ciclos de vida o metodologías tradicionales 28
5.3 Herramientas CASE (Computer Aided Software Engineering) 31
6. Proceso de obtención de código ejecutable a partir del código fuente, herramientas implicadas 32
6.1 Ejecución basada en complicación 32
6.2 Ejecución basada en interpretación 33

6.3 Ejecución basada en máquina virtual 34
7. Errores en el desarrollo de programas 36
8. Importancia de la reutilización de código 37
9. Resumen 38
Glosario 39
Bibliografía 41

Objetivos

- Conocer qué es un programa informático.
- Identificar los diferentes tipos de código.
- Clasificar los lenguajes de programación en función de sus características.
- Identificar las fases del desarrollo de aplicaciones y las formas de combinarlas.
- Aprender las diferentes alternativas existentes a la hora de obtener un programa ejecutable.

Tech Hub <FP> euroformac

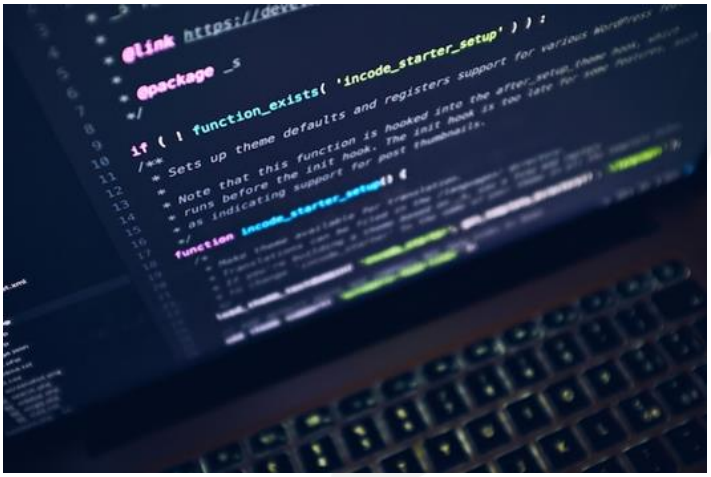
¿Todos los lenguajes de programación son iguales?

1. Introducción

El **desarrollo de software** es una actividad que incluye múltiples tareas. La elección del lenguaje de programación y de la metodología de trabajo, la selección de herramientas, la creación del propio código y obtención del programa ejecutable, son algunas de las más importantes. Todas estas tareas tienen una base teórica que es necesario conocer para poder realizarlas con criterio.

En esta unidad se presentan algunos de los conceptos y técnicas más relevantes relacionados con el desarrollo de software que permiten a los profesionales de la programación llevar a cabo su trabajo correctamente.

2. Concepto de programa informático



Un **ordenador** es una máquina compleja, formada por un conjunto de **componentes** capaces de realizar multitud de tareas en muy poco tiempo. Estas tareas van desde la lectura de datos de un disco duro, hasta la representación de una imagen en la pantalla, pasando por la evaluación de condiciones lógicas o la realización de operaciones aritméticas. De manera correctamente ordenada, una sucesión de estas tareas será capaz de resolver un problema. La principal dificultad radica en **cómo transmitir al ordenador qué se quiere hacer.**

El lenguaje humano es muy **flexible y complejo.** Estas características hacen que sea potencialmente impreciso, ambiguo. Por suerte, las personas disponen de la capacidad intelectual suficiente para resolver imprecisiones o ambigüedades con más o menos cierto.

Por ejemplo, cuando una persona le indica a otra que se coloque en determinado lugar de una estancia, no le indica qué tiene que hacer para ello: el destinatario de la orden comenzará a avanzar en dirección al destino por uno de los posibles caminos a una velocidad determinada hasta completar el objetivo. A priori, el ordenador no es capaz de resolver este tipo de problemas. Si un ordenador tuviese la mecánica de una persona; habría que indicarle paso a paso qué músculos mover, con qué intensidad y durante cuánto tiempo. Es por esto por lo que el lenguaje humano (conocido como lenguaje natural) no es apto para dar instrucciones a un ordenador.

Un **programa informático** es la herramienta para hacer esta transmisión de información. Se compone de una serie de **instrucciones**, escritas en un lenguaje formal, que son directa o indirectamente entendidas por el ordenador y convertidas en acciones concretas.

A pesar de que es posible, **la transmisión de instrucciones al ordenador de manera directa es muy compleja**. Requiere conocer al detalle sus componentes físicos para poder indicarles qué deben hacer en cada momento. Se puede afirmar que, de manera aislada, el hardware de un ordenador no tiene ninguna utilidad práctica. Sería lo mismo que un automóvil sin mecanismo de encendido, volante ni acelerador.

Sabías que

El *hardware* de un ordenador está compuesto por sus componentes físicos: el disco duro, la tarjeta de vídeo, el teclado, el ratón o la pantalla son algunos ejemplos.

Para poder utilizar un ordenador de una manera razonablemente sencilla es necesario contar con un sistema operativo. Un sistema operativo es capaz de "hablar" con el ordenador para decirle qué debe hacer: hace de intermediario entre el programa informático (creado por las personas) y la máquina física. A su vez, el sistema operativo es también un programa informático con unas características muy concretas.

Por lo tanto**, un programa informático es un conjunto de instrucciones escrito en un lenguaje de programación que tienen como objetivo realizar un conjunto de tareas en un ordenador utilizando el sistema operativo como intermediario en la comunicación con dicho ordenador.**

Además de símbolos, la mayoría de los lenguajes de programación están formados por palabras que forman parte del diccionario de la lengua inglesa directamente o a través de variaciones. Con esto se consigue simplificar tanto la escritura como la lectura y comprensión de los programas.

En el siguiente bloque se puede observar un programa escrito en lenguaje C. Aún sin conocer este lenguaje, se pueden identificar las palabras **include** (incluir), **main** (principal), **printf** (una variación de print, imprimir) y **return** (retorno). Como se puede suponer, estas palabras no son comprensibles por el ordenador si no es mediante un proceso de conversión, pero sí permiten hacerlo a las personas.

```
#include <stdio.h>
int main() {
   printf("Esto es un programa escrito en C");
   return 0;
}
```

A la acción de escribir un programa se la denomina **programar** y a los profesionales que realizan dicha actividad se les denomina **programadores.**

Existen multitud de lenguajes de programación. Algunos de los más más populares **son Java, Python, C++, C#, PHP, Ruby, JavaScript, Cobol o el citado Fortran.** Un programador suele utilizar varios lenguajes de programación a lo largo de su vida profesional, siendo importante el aprendizaje de los fundamentos de la programación para sentar una base teórica sólida sobre la que ampliar conocimientos sin demasiada dificultad.

Ilustración 1Margaret Hamilton, programadora de la misión espacial Apolo 11 de la NASA

Sabías que

FORTRAN fue el primer lenguaje de programación comercial de la historia, creado en 1957 y aún vigente en la actualidad.

3. Ejecución de programas en ordenadores

Los **programas informáticos** son una sucesión de ficheros que no son útiles hasta que se ejecutan. La **ejecución** es el proceso de cargar en el ordenador el programa y permitir al sistema operativo que le proporcione recursos para poder realizar sus tareas.

En la ejecución de los programas participan **varios actores**, como los propios programas y sus datos, los elementos físicos o los lógicos del ordenador, así como otros componentes necesarios para el funcionamiento de las computadoras.

3.1 Datos y programas

Cuando se habla de un programa informático y de la programación de este, **se suele pensar en líneas de código con instrucciones** en las que se indica al ordenador qué debe hacer. En cierto modo esto es correcto, pero no siempre es del todo completo.

Un programa es **un conjunto de líneas de código** que realizan una tarea, pero frecuentemente necesitan información para realizarla. Por ejemplo, un programa que sea capaz de realizar operaciones aritméticas como la suma o la resta deberá saber qué operandos utilizar. A toda esa información necesaria se le conoce en programación como dato.

Los datos pueden estar almacenados en un **dispositivo de almacenamiento** (un disco duro o un pendrive, por ejemplo), en la web (almacenamiento en la nube) o en la memoria del ordenador (almacenamiento volátil). Por su parte, el origen de los datos puede ser muy diverso: un usuario, un sensor de temperatura, un sitio web de información bursátil o un correo electrónico debidamente procesado.

3.2 Hardware y software

Una de las clasificaciones de más alto nivel de los elementos de un ordenador los divide en elementos **hardware** y **software.**

El conjunto de los elementos hardware está compuesto por todo aquello que es físico: discos duros, memorias RAM, pantallas, procesadores, etc.

Por su parte, **el software** está formado por los elementos lógicos e intangibles, "el alma" del ordenador. Todos los programas forman parte del software, incluido el sistema operativo. También forman parte del software los controladores (programas que permiten comunicarse con el hardware) y el **firmware**, un tipo de programa integrado en chips en muchos dispositivos hardware.

3.3 Estructura funcional de un ordenador: procesador, memoria

Los ordenadores modernos se basan, en esencia, en la conocida como **Arquitectura de Von Neumann,** un matemático nacido en el antiguo imperio austrohúngaro. Según la definición de Von Neumann, **los elementos de un ordenador son la Unidad Central de Proceso (CPU), la memoria principal y los dispositivos de entrada y salida.**

Más info

Se puede profundizar en el conocimiento de Von Neumann y su arquitectura en el siguiente enlace:

Ver enlace.

3.4 Tipos de software. BIOS. Sistema. Aplicaciones

Como se ha indicado en esta misma unidad, el software es un concepto que abarca a la parte intangible (sin incluir los datos) que forma parte del ordenador y que permite su funcionamiento.

El software se puede categorizar de la siguiente manera, **en función de su naturaleza:**

- **Software de sistema.** Permite el funcionamiento del ordenador: sistema operativo, controladores y firmware.
- **Software de aplicación.** Permite a los usuarios de los ordenadores realizar sus tareas: navegadores de internet, procesadores de texto, sistemas de contabilidad, videojuegos, etc.
- **Software de programación.** Es el utilizado por los programadores para la creación de programas: compiladores, enlazadores, intérpretes o máquinas virtuales son algunos ejemplos.

Dentro del firmware, se encuentran los casos concretos del **BIOS** (Basic Input/Output System) y del **UEFI** (Unified Extensible Firmware Interface). En ambos casos, contienen los programas configuran y verifican el buen funcionamiento del hardware antes de iniciar la ejecución del sistema operativo. El **UEFI** es la evolución del **BIOS**.

Más info

Se puede profundizar en el conocimiento del concepto de *BIOS* y *UEFI* en el siguiente enlace:

Ver enlace.

3.5 Código fuente, código objeto y código ejecutable, máquinas virtuales

Antes de escribir un programa informático hay que **elegir el lenguaje en el que se va a hacer**. Esta elección viene determinada por **diversos factores,** algunos más objetivos como la velocidad de ejecución que se desea obtener o más subjetivos como la experiencia del programador o programadores en uno u otro lenguaje.

Esta elección tiene implicaciones en el modo en el que los programas son traducidos y ejecutados en el sistema operativo al que va destinado dicho programa. A pesar de estas diferencias, es necesario conocer **algunos de los conceptos que participan en la creación, transformación y ejecución de los programas**. Estos son:

- Código fuente.
- Código objeto.
- Código ejecutable.
- Máquina virtual.

Algunos de estos conceptos **tienen validez en unos lenguajes de programación, pero no en otros**. Depende de determinadas características del lenguaje.

3.5.1 Código fuente

El primero de los términos que presentar es el de **código fuente**, existiendo este término en todos los lenguajes. Se conoce con este nombre al programa tal y como ha sido escrito por el programador. **No es directamente comprensible por el ordenador**, por lo que necesitará de algún proceso de conversión o traducción para conseguir que esto sea posible. El código fuente es la traducción del término en inglés ***source code***. A continuación, se muestran los códigos fuente de un mismo programa escrito **tres lenguajes de programación diferentes:** C, Java y Python.

- **C:**

```
#include <stdio.h>
int main() {
   printf("Hola mundo");
   return 0;
}
```

- **Java:**

```
package com.entornosdedesarrollo.ejemplo;

public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("Hola mundo");
    }
}
```

- Pyhton:

```
if __name__=='__main__':
    print("Hola Mundo")
```

Es evidente que existen múltiples diferencias entre los códigos fuente de cada lenguaje, pero también que hay algunas coincidencias y palabras comunes (**main, print** o alguna variación). En los tres casos, el código ha sido escrito por una persona, no es directamente comprensible por el ordenador, pero será convertido en algo que sí lo será. Son tres ejemplos de **código fuente**.

3.5.2 Código objeto y código ejecutable

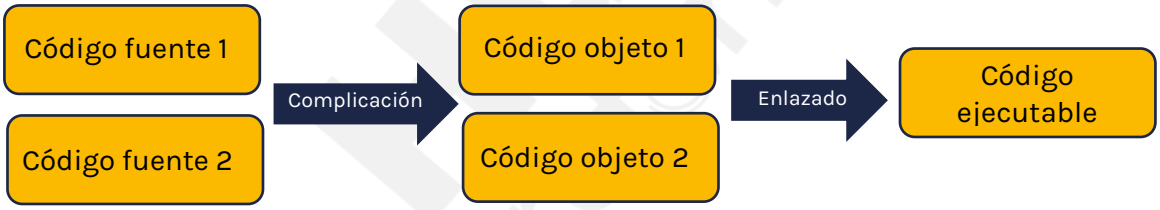
Como ya se ha indicado anteriormente, el código fuente está escrito en un lenguaje que no es entendible por el ordenador. **El ordenador tiene un lenguaje propio**, conocido como **lenguaje máquina** o **código máquina**. Como

su propio nombre apunta, las instrucciones expresadas en este lenguaje **son directamente ejecutables en el microprocesador del ordenador.**

Por lo tanto, es necesario **convertir el programa escrito por el programador en un lenguaje de programación (código fuente) al lenguaje del ordenador (código máquina).** Este proceso se conoce como **compilación** y es imprescindible conocer las razones de su existencia para presentar los conceptos **código objeto** y **código ejecutable**.

La diferencia entre ambos es sencilla: **el código objeto** no es directamente ejecutable pese a que contiene instrucciones que sí lo son. **El código ejecutable** sí es directamente ejecutable. La transformación del código objeto en ejecutable se realiza a través de un proceso conocido como enlazado (conocido informalmente como *linkado*, de la palabra inglesa link), mediante el cual se construye un programa ejecutable a partir de uno o más códigos objetos.

Es importante saber que **el proceso de enlazado no existe en todos los lenguajes de programación ni es obligatorio en todos los casos**. Es posible generar un código ejecutable a partir de un código fuente sin pasar por este proceso.

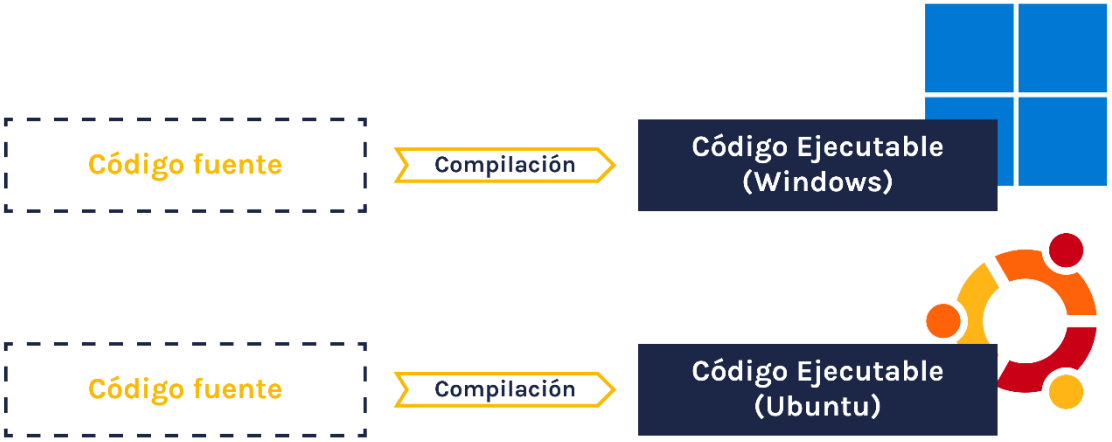


Tanto el código objeto como el código ejecutable son el resultado de transformar el código fuente de un programa en una versión comprensible (ejecutable) por el ordenador a través del **sistema operativo**. Esta afirmación tiene una importante consecuencia: **tanto el código objeto como el código ejecutable es únicamente válido para un sistema operativo concreto (o alguno compatible).** Por ejemplo, un código ejecutable para el sistema operativo Windows 10 no será ejecutable en Linux ni en MacOS. Tampoco será ejecutable en Windows 95 pero probablemente sí lo sea en Windows 11. En resumen, el código ejecutable será válido para el sistema operativo para el que se construyó y para sistemas operativos compatibles.

Importante

Un programa ejecutable lo es solo para la plataforma (sistema operativo) de destino.

Esta restricción no significa que siempre haya que escribir un programa diferente para cada sistema operativo, aunque a veces sea necesario. Lo que implica esta restricción es **que hay que realizar un proceso de compilación diferente para cada sistema operativo** en el que se quiera ejecutar el programa.



Sabías que

El proceso de compilación es una sucesión de actividades que tiene como principal objetivo la obtención del programa ejecutable. Previamente, el compilador verifica que el código fuente está escrito correctamente (desde los puntos de vista léxico, sintáctico y semántico), avisando al programador si ha cometido algún error para que lo solucione antes de continuar. **Durante la compilación, por lo tanto, se verifica que el código es correcto y genera el código ejecutable.**

3.5.3 Bytecode y las máquinas virtuales

En el mejor de los casos, tener que compilar un programa tantas veces como sistemas operativos en los que se quiera ejecutar requiere una inversión de tiempo y recursos. En el peor de los casos, los programas deben ser **revisados y modificados** para los diferentes sistemas si se utilizan elementos específicos de dichos sistemas operativos. Parece evidente que **esta es una molestia que convendría evitar.**

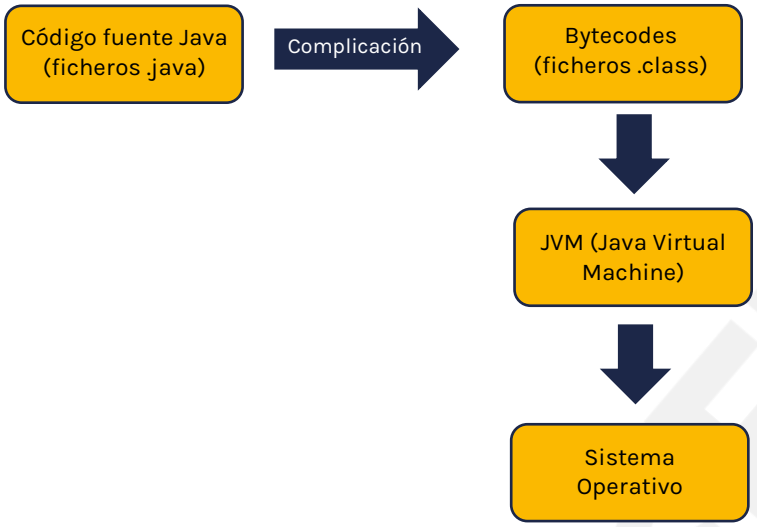
Existe una **solución** a este problema basado en la utilización de máquinas virtuales. En lo referente a los programas informáticos, **una máquina virtual** es un **programa informático que simula ser un ordenador** capaz de ejecutar programas. Desde dentro de la máquina virtual no se ve el ordenador en el que se está ejecutando esta, sino la propia máquina y sus recursos, utilizando su propio lenguaje máquina.

Sabías que

Una de las características más relevantes de la ejecución de programas basadas en máquinas virtuales es la seguridad.

Cuando se programa para una máquina virtual, el código fuente se transforma en un "*código ejecutable*" destinado a la misma pero que no es ejecutable en ninguna máquina "real", sino únicamente en dicha máquina virtual. A este código de destino se le conoce con el nombre de **bytecode**. Por lo tanto, se puede definir como **bytecode al lenguaje entendible que por una máquina virtual**. Es la máquina virtual la que se encarga de traducir los bytecodes al código binario entendible por la máquina real en la que se encuentra ejecutándose.

El lenguaje Java es, probablemente, el ejemplo más característico de uso de máquinas virtuales en programación. Cuando se escribe un programa en Java este es compilado y traducido a una versión en *bytecodes*. Para poder ejecutar esta traducción se necesita tener instalado en el ordenador una máquina virtual (un programa) que entienda los *bytecodes* y que sea capaz de traducir sus instrucciones al lenguaje máquina del ordenador en el que se encuentra instalada. Por lo tanto, el mismo programa Java traducido a *bytecodes* será potencialmente ejecutable en cualquier ordenador siempre que este tenga instalada la máquina virtual correspondiente. En el caso de Java, esta máquina se llama *Java Virtual Machine* (JVM) o Máquina Virtual de Java.



Crear una máquina virtual y su propio lenguaje máquina tiene como uno de sus objetivos principales **conseguir la portabilidad de los programas**, ya que únicamente será necesario tener instalada la máquina virtual en un ordenador con cualquier sistema operativo, permitiendo disponer de aplicaciones multiplataforma.

En algunos lenguajes interpretados, como **Python o PHP, el código fuente se transforma en *bytecodes*** que son traducidos a código máquina por el intérprete, de manera similar a como lo hace una máquina virtual.

4. Lenguajes de programación

Un **lenguaje de programación** es un lenguaje formal que tiene la capacidad de expresar instrucciones capaces de ser transformadas y transmitidas a un ordenador con el objetivo de que este realice una tarea.

4.1 Tipos de lenguajes de programación

Existen cientos de lenguajes de programación, cada uno de ellos con sus características y particularidades.

Más info

En el siguiente artículo se enumeran algunos de los lenguajes de programación más utilizados en la actualidad.

Ver enlace.

Como ocurre con cualquier gran conjunto de elementos, es necesarios establecer tipos para poder organizarlos de manera sencilla. En lo referente a los lenguajes de programación las clasificaciones son múltiples. Algunas de las más relevantes se muestran a continuación.

4.1.1 Por un nivel de abstracción y generación

Agrupan los lenguajes en función de características como el **nivel de abstracción** y, a partir de este, **por generación.**

El **nivel de abstracción** hace referencia a cuánto debe programarse teniendo en cuenta las características de la máquina (conocimiento interno de la misma).

El **número de generación** indica la secuencia de aparición de los primeros lenguajes de dicha generación, pero no

obliga a que un lenguaje moderno sea de la última generación.

- **Bajo nivel**. Son absolutamente dependientes de la máquina ya que los programas escritos en este tipo de lenguaje acceden a sus componentes lógicos (memoria o registros del procesador, por ejemplo) de manera directa.
    - **Primera generación**. Programación directa en código máquina o binario (ceros y unos). Una sentencia escrita en código máquina tendrá un aspecto similar al siguiente: **000000 00001 00011 00100 00000 100000.**
    - **Segunda generación.** Programación en ensamblador (una abstracción de mayor nivel del código máquina), compuesto por instrucciones que contienen una simbología y códigos legibles por las personas. Estas instrucciones son convertidas a código máquina mediante un ensamblador. Un ejemplo de sentencia escrita en ensamblador es la siguiente: **MOV DX, AX.**

- **Alto nivel.** Utilizan una notación textual o simbólica comprensible por las personas. No requiere conocer el funcionamiento interno de la máquina ya que las instrucciones son traducidas mediante compiladores o intérpretes.
    - **Tercera generación**. Lenguajes de alto nivel, con sintaxis más sencilla y comprensible. Algunos ejemplos son Java, JavaScript, C, C++ o C#.
    - **Cuarta generación.** Lenguajes especializados en dominios concretos, como el manejo de datos, la generación de informes o el desarrollo de aplicaciones web o con interfaz gráfica de usuario. En algunos casos, suelen proporcionar facilidades para la programación, incluyendo herramientas visuales que permiten la creación de programas sin necesidad de escribir código.
    - **Quinta generación**. La programación se basa la expresión de reglas en un lenguaje similar al natural y no en algoritmos e instrucciones convencionales. Prolog es el ejemplo más característico.

Lenguajes como **Python, PHP, Perl o Ruby son indudablemente de alto nivel**, pero existen discrepancias entre si pertenecen a la tercera o cuarta generación, ya que tienen características propias de ambas agrupaciones.

Sabías que

La clasificación por generaciones no está exenta de discrepancias. Dependiendo de la fuente, puede estar compuesta por 5 o 6 generaciones. También puede haber lenguajes que se encuentren en una u otra generación dependiendo del autor. Estas diferencias son habituales en cualquier clasificación.

4.1.2 Por paradigma

En programación, **el paradigma** hace referencia a **cómo se estructura un programa y sus componentes**. El paradigma implica una forma de resolver los problemas. Hay que tener en cuenta que **los paradigmas de programación no son mejores ni peores,** sino más o menos aptos para resolver determinados problemas. Los principales paradigmas de programación son los siguientes:

- Paradigma **imperativo**. En los lenguajes imperativos, el programa contiene la sucesión de pasos ordenados que han de realizarse para la resolución de un problema. C o Pascal son ejemplos de lenguajes de este tipo de paradigma.
- Paradigma **declarativo.** En los lenguajes declarativos el programa contiene información sobre cómo debe ser la solución y no los pasos para llegar a ella. PROLOG es uno de los lenguajes más representativos de este tipo de paradigma.
- Paradigma de **programación estructurada**. Es un subtipo de la programación imperativa. Estructura los programas en agrupaciones de código fuente conocidas como funciones o procedimientos, facilitando la organización y la reutilización del código. Utiliza estructuras de control para determinar el flujo de ejecución. C es un ejemplo de este tipo de programación.

- Paradigma **orientado a objetos**. Es un subtipo del paradigma imperativo en el que el programa se construye a partir de conceptos como la clase y el objeto, que son entidades que agrupan elementos del programa en unidades reutilizables. Aunque inicialmente este paradigma pueda resultar complejo, cuando se domina permite encontrar buenas soluciones a prácticamente cualquier tipo de problema. C#, C++, Java, Python o PHP, por citar algunos ejemplos, permiten la programación con este paradigma.

- Paradigma **funcional**. Es un subtipo del paradigma declarativo en el que se estructura la solución de los problemas a través de funciones matemáticas. Algunos lenguajes que no son puramente declarativos tienen capacidades de programación funcional, como C#, C++ o Java.

Existen muchos otros paradigmas, más o menos importante. Por ejemplo, la **programación reactiva**, enfoca la solución a los problemas desde la perspectiva de los flujos de datos que se producen en los sistemas.

Los **lenguajes de programación pueden estar enfocados a uno o más paradigmas.** Java, por ejemplo, es un lenguaje puramente orientado a objetos. C, por su parte, es puramente estructurado. Python, en cambio, admite los paradigmas declarativos, orientado a objetos y funcional. A este tipo de lenguajes se le conoce como **multiparadigma.**

4.1.3 Por forma de ejecución

Otra importante clasificación organiza los lenguajes de programación **a partir de la forma en que son ejecutados, existiendo tres alternativas** principales diferentes.

- **Lenguajes de programación compilados**. Los programas escritos en este tipo de lenguajes deben ser compilados (traducidos a código máquina) como paso previo a su ejecución. El proceso de compilación se debe realizar de manera individual para cada sistema operativo de destino. La principal característica y ventaja de este tipo de lenguajes es que, una vez compilado, el programa se puede ejecutar de manera directa, obteniendo mayor velocidad de ejecución. Otra ventaja es que una vez creado el ejecutable, no se necesita de ningún elemento de software ajeno para su ejecución. C y C++ son buenos ejemplos de este tipo de lenguajes.

- **Lenguajes de programación interpretados**. En este tipo de lenguajes, los programas no son compilados para obtener un ejecutable, sino que sus instrucciones son traducidas durante la ejecución al lenguaje

máquina. Para ello se necesita un software de traducción, conocido como **intérprete**, que deberá estar instalado en cada ordenador en el que se quieran ejecutar los programas escritos con este tipo de lenguajes. La principal ventaja es la portabilidad, ya que el código fuente puede ser ejecutado en cualquier plataforma si esta cuenta con el intérprete adecuado. Python y PHP son ejemplos de lenguajes de programación interpretados.

- **Lenguajes basados en máquinas virtuales**. El código fuente de los programas escritos con estos lenguajes es compilado, pero no para una máquina real, sino para una virtual (un sistema software que funciona como un ordenador dentro de otro ordenador). Al igual que en los lenguajes interpretados, es necesario que la máquina virtual esté instalada en el ordenador real para poder ejecutar el resultado de la compilación. Es la máquina virtual, por lo tanto, quien se encarga de traducir el código compilado (conocido como *bytecode*) al lenguaje máquina comprensible por el ordenador. Una ventaja de este tipo de lenguajes es la portabilidad, al igual que en los lenguajes interpretados. Otra ventaja es la seguridad, ya que al ejecutarse en una máquina virtual (un entorno cerrado) es sencillo limitar el acceso a los recursos de la máquina real. Desde el punto de vista del rendimiento, estos lenguajes se encuentran en un punto intermedio entre los compilados y los interpretados.

Existen lenguajes que se encuentran entre medias de algunas de estas categorías. JavaScript, por ejemplo, es considerado frecuentemente como un lenguaje interpretado, pero internamente los programas son compilados y ejecutados en un motor que guarda bastantes analogías con una máquina virtual.

Por otra parte, **los programas escritos en lenguajes** como Python y PHP, considerados como lenguajes interpretados, **son convertidos a *bytecodes* mediante un proceso de compilación** al igual que los lenguajes basados en máquinas virtuales. En este caso, el intérprete es el encargado de traducir los *bytecodes* a código máquina.

También es importante saber que **muchos de los lenguajes interpretados disponen de herramientas para compilar el código fuente y obtener versiones ejecutables**. Esta acción elimina la posibilidad de ejecutarse en múltiples plataformas, pero permite reducir los tiempos de ejecución.

4.2 Características de los lenguajes difundidos

Los lenguajes de programación tienen ciertas características que los definen y que es necesario conocer.

Antes de comenzar la exposición es necesario conocer los conceptos **de variable y de tipo de variable,** ya que participan de algunas de las características de los lenguajes.

Una variable es un espacio de memoria en el que se almacena un dato. Para identificar la variable **se le asigna un nombre,** conociendo al dato almacenado como valor. Por ejemplo, la variable edad puede almacenar el valor 25, mientras que la variable nombre puede almacenar el valor "*Marta*". Se puede establecer una analogía entre una variable y una nota adhesiva en la que se puede escribir un dato para evitar olvidarlo.

El **tipo de variable** hace referencia a la **naturaleza del dato que es capaz de almacenar**. Una variable de tipo numérico podrá almacenar números mientras que una de tipo alfanumérico podrá almacenar tanto caracteres alfabéticos como dígitos numéricos.

Por último, hay que indicar que existen **dos acciones que tienen relación con las variables:** la **declaración y la asignación**. La declaración consiste en indicar el nombre que va a tener la variable y, dependiendo del lenguaje, su tipo. La asignación consiste en indicar qué valor tiene la variable. En el código de ejemplo siguiente, escrito en lenguaje C, se muestra la declaración (primera línea) de la variable de tipo entero (*int*) edad y la asignación (segunda línea) del valor 25 a la misma:

```
int edad;
edad=25;
```

A continuación, se exponen las características más relevantes de los lenguajes de programación.

- **Nivel de abstracción**. Un lenguaje de programación puede ser de **alto o bajo nivel**. A más alto nivel, más sencilla de comprender es la sintaxis requiriendo menos conocimiento del funcionamiento interno del ordenador.
- **Paradigma**. Refiere al enfoque que acepta el lenguaje para elaborar una solución. **Imperativo, declarativo, estructurado, orientado a objetos** o **funcional** son algunos de los principales paradigmas.
- **Plataforma de destino**. Hace referencia al sistema operativo sobre el que se pueden ejecutar los programas. En lenguajes compilados, la existencia del compilador específico para cada sistema determinará si admite dicha plataforma. Los principales lenguajes interpretados o de máquina virtual, se consideran **multiplataforma.**
- **Tipo de ejecución**. Compilado, interpretado o basado en máquina virtual.
- **Tipado fuerte o débil**. Se dice que un lenguaje es **fuertemente tipado** cuando el tipo de sus variables determina qué acciones se pueden realizar con él, impidiendo aquellas que no le son propias. Por ejemplo, un lenguaje fuertemente tipado no permitirá dividir una variable de tipo alfanumérico entre un valor numérico salvo que disponga de una operación específica para hacer dicho tipo de operaciones. Java o C++ son ejemplos de lenguajes fuertemente tipados. Los lenguajes **débilmente tipados** o **no tipados** permiten hacer uso indiscriminado de las variables sin importar su tipo. JavaScript es un ejemplo de lenguaje débilmente tipado.
- **Tipado estático o dinámico**. En el tipado estático, al declarar el nombre de la variable se indica el tipo y este estará vinculado a dicha variable durante toda la ejecución del programa. Los lenguajes C, C++ o Java utilizan tipado estático. En el tipado dinámico, el tipo de la variable se asigna como consecuencia de la asignación de un valor (si se asigna un número entro, la variable es de tipo entero, por ejemplo), pudiendo cambiar el tipo durante la ejecución del programa. Los lenguajes JavaScript o Python son ejemplos de este tipo de lenguaje.
- **Licencia**. La licencia hace referencia a cómo se puede utilizar el lenguaje de programación y si es necesario pagar algún tipo de canon

a su propietario. Habitualmente los lenguajes de programación se pueden utilizar sin costo, al igual que ocurre con los compiladores, las máquinas virtuales o los intérpretes, aunque existen excepciones. Un tipo de licencia particular es la conocida como de **código abierto**, cuyo propósito es permitir el acceso y uso de su definición e implementación (compilador), pudiendo crear sin restricciones compiladores u otras herramientas. Java, PHP o Python son ejemplos de lenguajes de código abierto.

- **Propósito.** Indica para qué está diseñado y concebido el lenguaje. Puede estar orientado a la resolución de problemas de lógica (por ejemplo, PROLOG), negocios (por ejemplo, COBOL) o matemáticas e ingeniería (FORTRAN es un ejemplo). Los lenguajes más populares suelen ser de **propósito general**, lo que indica que pueden ser útiles para la resolución de cualquier tipo de problema, lo que no indica que no estén especializados en algún área concreta. Python, Java, C#, C, C++ o PHP son ejemplos de este tipo de lenguajes.

Más info

Se puede profundizar en el conocimiento del concepto de código abierto en el siguiente enlace:

Ver enlace.

5. Introducción al Ingeniería del Software

Las técnicas de gestión del desarrollo de aplicaciones, incluyendo tanto las fases como las metodologías, se encuentran englobadas en una disciplina profesional y académica conocida como **ingeniería del software**.

Por lo tanto, se puede definir a **la ingeniería del software como la disciplina que se encarga de definir las técnicas, metodologías y herramientas necesarias para desarrollar correctamente las aplicaciones informáticas.**

5.1 Proceso del software y ciclo de vida del software

Una aplicación informática es un conjunto de uno o más programas que ayudan a realizar una tarea o a resolver un problema. Existen, como es obvio, aplicaciones de diferentes tamaños, desde pequeñas y sencillas, como pueda ser un conversor de euros a dólares, hasta increíblemente grandes y complejas, como pueda ser un sistema operativo.

Las **aplicaciones informáticas** deben ser creadas, puestas en funcionamiento o mantenidas, al igual que se haría con cualquier tipo de herramienta o maquinaria física. Un vehículo, por ejemplo, debe ser concebido, diseñado, creado, puesto en marcha y mantenido. Esto último incluye desde el cambio de piezas que se estropean por el uso hasta la sustitución de líquidos que se han degradado con el tiempo, pasando por el arreglo de los daños provocados por un accidente.

Con las aplicaciones informáticas ocurre algo parecido, estando **las diferentes fases de su ciclo de vida perfectamente identificadas**.

Se conoce como **ciclo de vida** de un programa o aplicación al conjunto ordenado de pasos que deben llevar a cabo desde su concepción hasta el final de su vida útil.

5.2 Fases del desarrollo de una aplicación: análisis, diseño, codificación, pruebas, documentación, explotación y mantenimiento, entre otras

En cualquier dominio, el ciclo de vida se define como un conjunto de fases y de las relaciones que hay entre ellas. En el desarrollo de software ocurre de la misma manera**: el ciclo de vida del desarrollo** está compuesto por la sucesión de fases y el orden en el que se deben llevar a cabo.

5.2.1 Fases del ciclo de vida

Las **fases tradicionales** del ciclo de vida de una aplicación informática son las siguientes:

- **Análisis.** En esencia, un sistema software satisface una necesidad. En la fase de análisis, se determina el alcance de solución de dicha necesidad: define de forma exhaustiva qué es lo que va a hacer el sistema software. En la fase de análisis, los profesionales conocidos como **analistas** recaban información sobre el sistema que hay que desarrollar. Mediante entrevistas y reuniones con los clientes y usuarios del sistema, lectura de documentación y estudio de los procesos, **en la fase de análisis se define qué es lo que va a hacer el sistema**.

- **Diseño.** En esta fase se determina **cómo se va a realizar la aplicación informática**. Consiste, entre otras cosas, en decidir cómo se va a organizar el proyecto a nivel de software: qué componentes van a crearse, como se van a comunicar entre ellos, qué dependencias existen o qué lenguajes de programación se van a utilizar. El diseño también incluye la definición de la base de datos y de las interfaces gráficas de usuario, siendo por lo tanto una actividad con múltiples enfoques. El perfil profesional que realiza este trabajo se conoce como **diseñador de software**.

- **Codificación.** Esta es la fase en la que se programa o codifica (son sinónimos) la aplicación. La codificación la realizan los

programadores, escribiendo el código manualmente o ayudándose de herramientas de generación de código.

- **Pruebas.** Esta fase consiste en garantizar que el sistema funciona de acuerdo con las especificaciones funcionales (qué hace) y técnicas (cómo lo hace). Las pruebas en el desarrollo de aplicaciones constituyen una actividad de extrema importancia ya que garantizan la calidad del software. La fase de pruebas es transversal, pudiéndose realizar sobre el análisis, el diseño, la codificación y, en ocasiones, sobre el resto de las fases.
- **Documentación.** Una aplicación informática puede ser extremadamente compleja, ya sea por el tamaño o por la dificultad de las tareas que realiza. Además, es un sistema vivo: debe ser corregido, mejorado y actualizado constantemente para que siga siendo útil. Para poder entender y mantener un sistema software es conveniente documentar todo el ciclo de vida, desde la fase de análisis hasta las pruebas, pasando por el diseño y la codificación. Por lo tanto, la fase de documentación en realidad es una fase transversal, teniendo presencia en el resto de las fases.
- **Explotación.** Esta fase ocupa las actividades que tienen que ver con el tiempo en el que la aplicación está en uso (explotación).
- **Mantenimiento.** El mantenimiento de los programas consiste en la resolución de los posibles errores (**mantenimiento correctivo**) y en la mejora de los elementos que funcionan (**mantenimiento evolutivo**). El mantenimiento se desarrolla durante todo el tiempo de la explotación del sistema.

Hay que saber que **estas fases no son estancas y que en muchas ocasiones se solapan o agrupan parcialmente.** Por ejemplo, las tareas de documentación se pueden desarrollar durante el resto de las fases, no siendo, por lo tanto, una tarea aislada del resto. Por su parte, las fases de análisis y diseño suelen solaparse, ya que se pueden realimentar. De hecho, las diferentes fases del desarrollo de software se pueden ordenar de diversas maneras. A **la forma en la que se ordenan las diferentes fases del desarrollo se conoce como metodologías o ciclo de vida**.

Dentro de las metodologías, existe un subconjunto que conocido como **metodologías ágiles**. Estas metodologías se caracterizan por **abordar la realización del proyecto desde un punto de vista iterativo y/o incremental,** en lugar de hacerlo desde una perspectiva completa, mucho más difícil de gestionar y controlar.

5.2.2 Ciclos de vida o metodologías tradicionales

Existe un **conjunto de ciclos de vida o metodologías tradicionales** que organizan las fases del ciclo de vida de distintas maneras. **Estas metodologías tienen más o menos encaje en los proyectos de desarrollo en función de su dificultad.**

Hay que considerar que los lenguajes de programación, las herramientas e incluso la formación de los programadores han ido evolucionando con el tiempo, por lo que constantemente se necesitan nuevas formas de trabajar: no es lo mismo desarrollar una aplicación en COBOL que en Python o en Java.

Ejemplos de las metodologías tradicionales son los ciclos de vida **en cascada, en V, iterativo, incremental** y **espiral.**

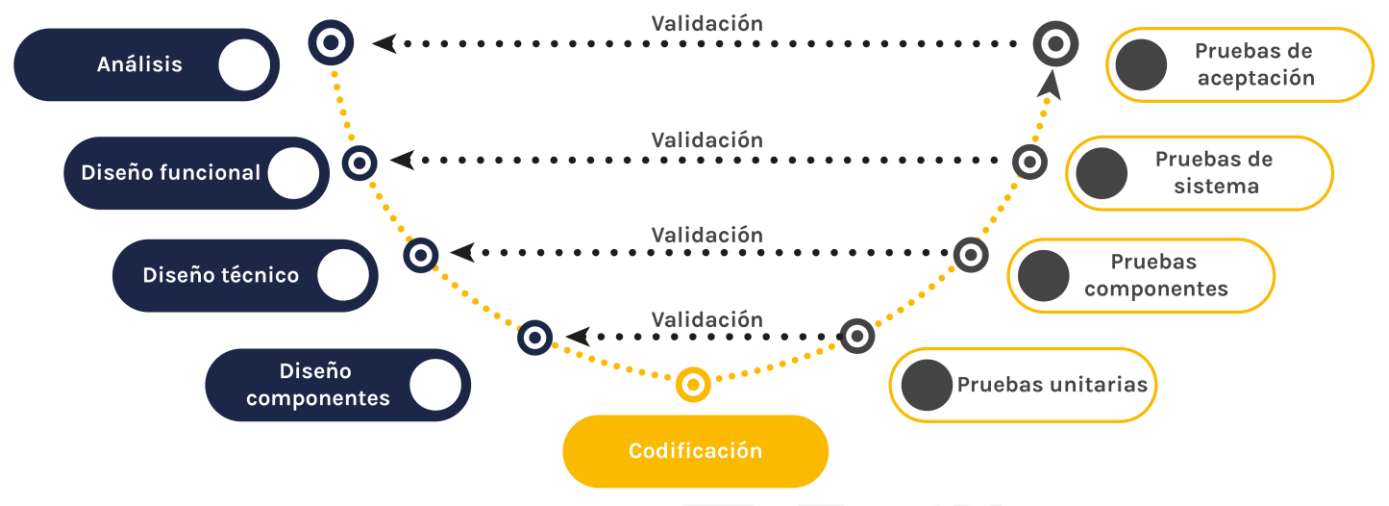
Algunas de estas metodologías se han confirmado como poco flexibles y apropiadas para proyectos dinámicos y expuestos a cambios funcionales y técnicos. Las metodologías ágiles tratan de ayudar a llevar a cabo este tipo de proyectos.

- **Modelos de proceso de desarrollo de software (cascada, iterativo, evolutivo)**

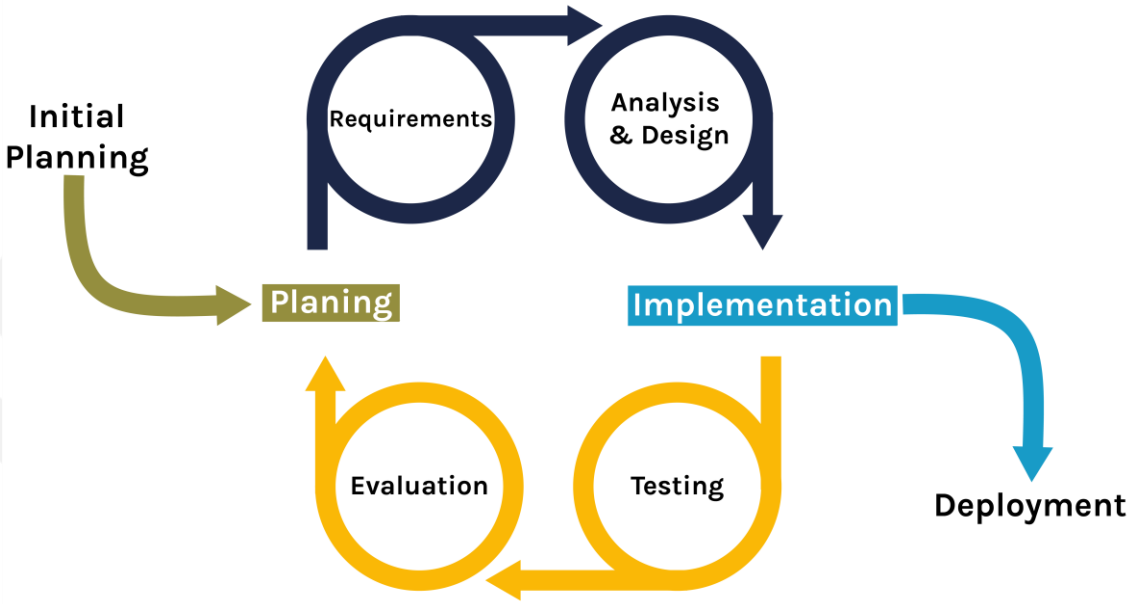
A continuación, se presentan algunos de los ciclos de vida de desarrollo de software tradicionales más populares:

- **Ciclo de vida en cascada.** El ciclo de vida más tradicional, conocido como desarrollo en cascada, propone la ejecución de las fases anteriormente citadas en orden secuencial. La propuesta del ciclo de vida en cascada fue aceptada en el pasado como válida, pero ha sido rechazada posteriormente por ser demasiado rígida. Un error en el análisis puede ser arrastrado por el resto de las fases, aumentando su impacto y el coste de su corrección.

- **Ciclo de vida en V**. Es una modificación del ciclo de vida en cascada en la que se definen específicamente las fases de verificación que garantizan la calidad del sistema.



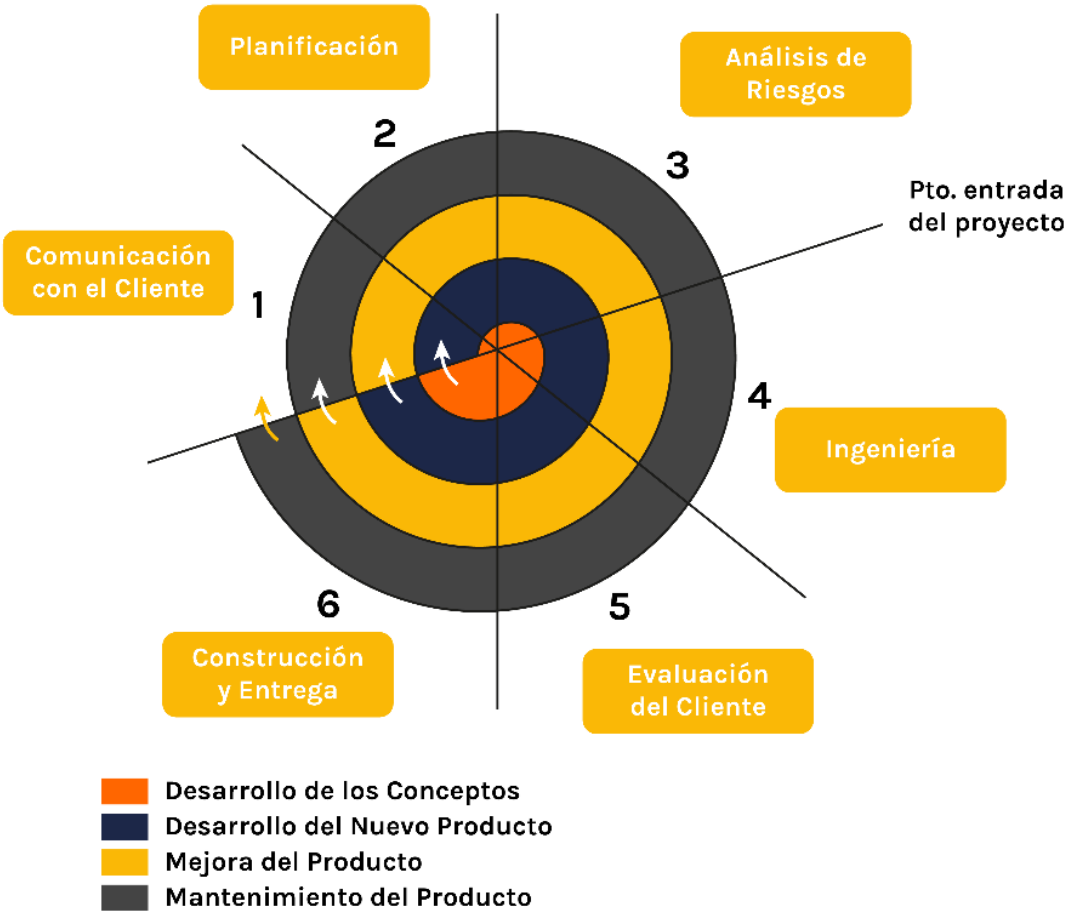
- **Ciclo de vida iterativo**. Esta forma de trabajar se considera metodología ágil. El ciclo de vida iterativo consiste, como su propio nombre indica, en realizar iteraciones sobre el proyecto para, en cada una de ellas, obtener una versión más detallada, optimizada y próxima a la versión final.



En el ciclo de vida iterativo los ciclos de desarrollo son cortos

- **Ciclo de vida incremental**. En este ciclo de vida el proyecto se divide en unidades más pequeñas y se abordan dichas unidades como proyectos completos. En cada versión (en cada iteración), el proyecto se ve incrementado con la adicción de la nueva unidad desarrollada. Con este enfoque, el proyecto se puede gestionar de manera más sencilla que si fuese una única actividad. Esta forma de trabajo también se considera una metodología ágil y comparte tantas características con el ciclo de vida iterativo que en ocasiones se fusionan ambas formas de trabajo.

- **Ciclo de vida en espiral**. Es una combinación entre el modelo en cascada y el basado en iteraciones. En este ciclo de vida, el alcance de cada iteración tiene relación con el análisis de los riegos, con el fin de detectar los problemas en el momento más temprano posible. En cada una de las iteraciones se construye nueva funcionalidad que se va incorporando al proyecto global.



5.3 Herramientas CASE (Computer Aided Software Engineering)

Las **herramientas CASE**, son aquellas herramientas que se utilizan para ayudar al desarrollo de aplicaciones automatizando parte de las tareas que se tienen que llevar a cabo. **Pueden estar presentes en prácticamente todas las fases del ciclo de vida,** desde el análisis hasta las pruebas, pasando por la programación (mediante la generación automática de código). Tienen como objetivos principales incrementar la productividad y la calidad de las aplicaciones.

Tech Hub <FP> euroformac

6. Proceso de obtención de código ejecutable a partir del código fuente, herramientas implicadas

Como se ha podido leer anteriormente, la generación de una versión ejecutable del código fuente **depende en gran media del tipo de lenguaje**. Se presentan, a continuación, los procesos de construcción y ejecución de un programa sencillo en tres lenguajes con distintos mecanismos de ejecución.

Todas las alternativas de compilación y/o ejecución de programas que se muestran a continuación **se realizan desde la línea de comandos**, ya que es la manera en la que se puede observar de forma explícita como se realizan estos procesos. En todos los casos, también **se podría realizar desde editores de texto configurados adecuadamente o mediante el uso del IDE (Entorno de Desarrollo Integrado o *Integrated Development Environment* en inglés) adecuado.**

6.1 Ejecución basada en complicación

Para poder ejecutar un programa escrito en un lenguaje de programación basado en la compilación **es necesario crear el programa ejecutable** (también conocido como binario) **mediante el proceso de compilación**.

Para ilustrar este proceso se utiliza el lenguaje C. Para escribir el programa bastará cualquier editor de texto, como por ejemplo *Visual Studio Code*. **Como compilador** se utilizará **GCC**, programa que se distribuye bajo Licencia Pública General de *GNU* (*GNU* General Public License), de uso gratuito y de código abierto.

El compilador GCC viene instalado por defecto en los sistemas Linux y Mac, pudiéndose obtener e instalar en los sistemas Windows a través del enlace siguiente: Ver enlace.

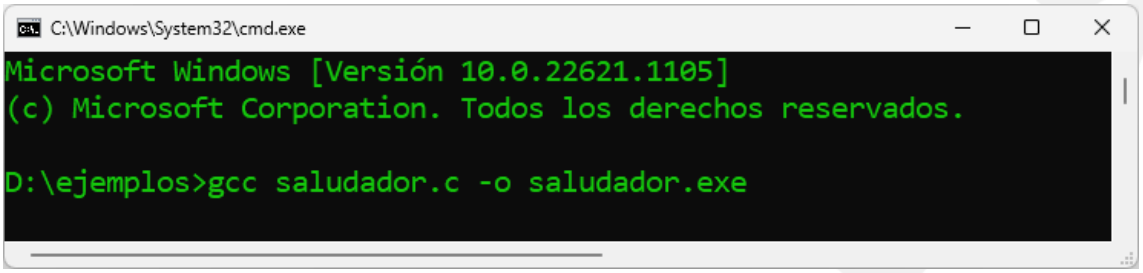
El **código fuente** debe escribirse en un fichero que, en el caso del lenguaje C, suele tener la extensión **.c**.

simulador.c

```
#include <stdio.h>
int main() {
   printf("Hola a todos");
   return 0;
}
```

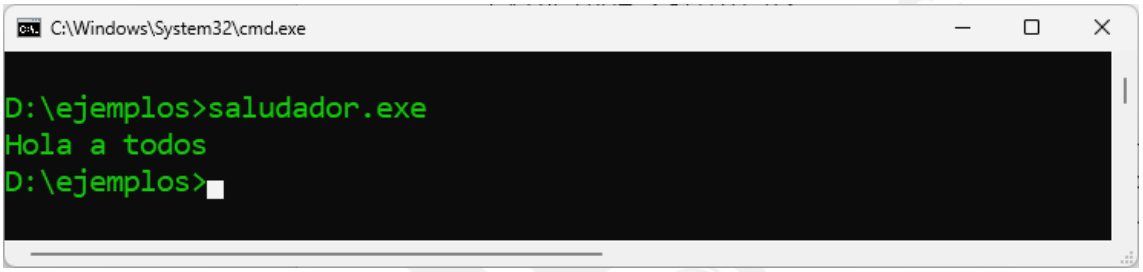
Con el compilador instalado y ubicado en la carpeta en la que se encuentra el fichero fuente, se debe ejecutar el proceso de compilación para verificar que la sintaxis es correcta y obtener el fichero ejecutable.

En la siguiente captura se puede observar la ejecución del programa de compilación (**gcc**) al que se le indica el nombre del fichero fuente a compilar (**saludador.c**), y el nombre del fichero de salida (parámetro **-o** y **saludador.exe**)



Compilación de un programa en C con GCC

El resultado es un fichero con el nombre **saludador.exe** que podrá ser ejecutado directamente en el sistema operativo para el que se ha compilado (en este caso para Windows 11).



Ejecución de un fichero binario

6.2 Ejecución basada en interpretación

Para ejecutar un programa escrito en un lenguaje interpretado es necesario **tener instalado el intérprete correspondiente.**

Por ejemplo, para ejecutar un programa escrito en Python, es necesario disponer del intérprete de Python en el ordenador en el que se va a realizar dicha ejecución.

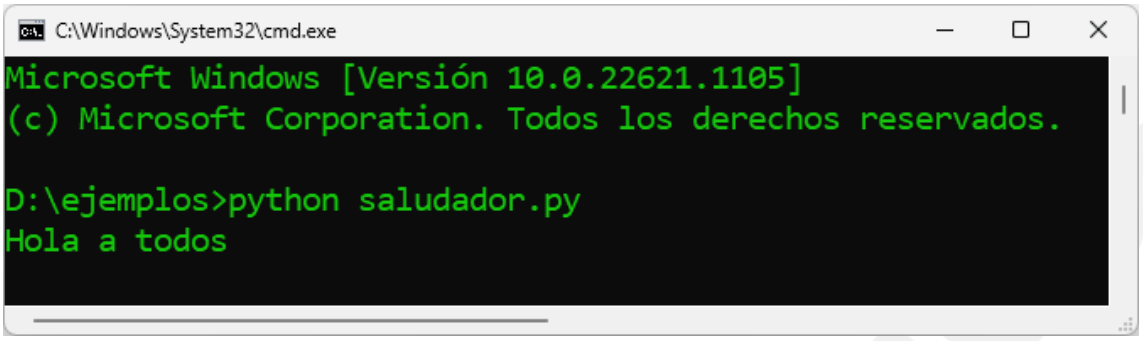
Python se puede obtener e instalar desde el siguiente enlace: Ver enlace.

El código fuente del programa a ejecutar deberá escribirse en un editor de texto como *Visual Studio Code* y almacenar en un fichero con la extensión **.py**.

saludador.py

```
print("Hola a todos")
```

En este tipo de lenguajes no hay que crear un fichero ejecutable. Simplemente se ejecuta el intérprete (programa **python.exe**) indicando a continuación el nombre del programa a ejecutar.



Ejecución de un programa escrito en Python

6.3 Ejecución basada en máquina virtual

El proceso de generación y ejecución de un *bytecode* (programa ejecutable en una máquina virtual) tiene aspectos comunes a los lenguajes compilados y a los interpretados.

En Java, por ejemplo, se necesita disponer de un compilador (programa **javac.exe**) y de un lanzador de la máquina virtual (programa **java.exe**). En el caso de este lenguaje concreto, la instalación de estas herramientas se realiza mediante la instalación del conocido como JDK (Kit de Desarrollo de Java o Java Development Kit), que se puede obtener en el siguiente enlace: Ver enlace.

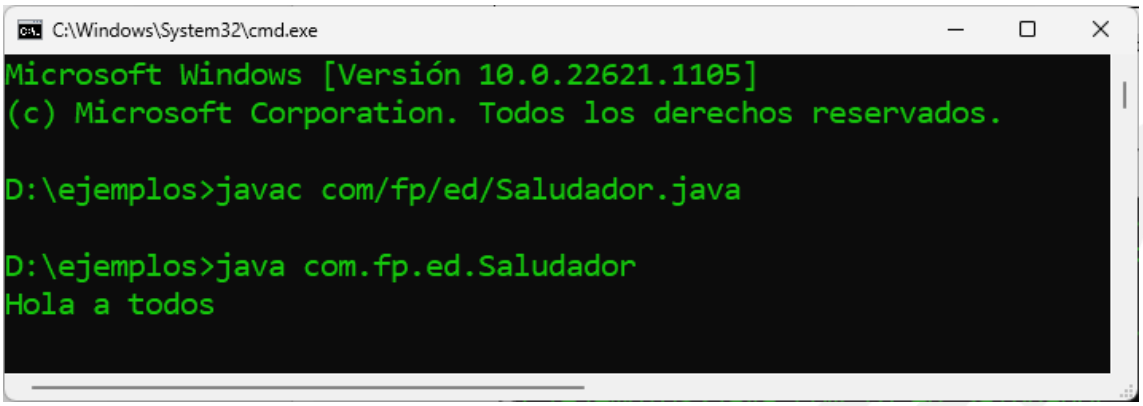
Los ficheros fuente Java tienen la extensión **.java** mientras que los ficheros compilados (*bytecode*) tienen la extensión **.class.**

Java organiza los ficheros fuente de una manera particular, en carpetas llamadas **paquetes**. En el ejemplo, el fichero tiene como nombre **Saludador.java** almacenada en la ruta **com/fp/ed**.

Saludador.java

```
package com.fp.ed;
public class Saludador {
    public static void main(String[] args) {
        System.out.println("Hola a todos");
    }
}
```

En la siguiente captura se puede observar el proceso de compilación (ejecución del programa **javac**) y de arranque de la máquina virtual (**java**) a la que se indica el programa Java que se quiere ejecutar (**com.fp.ed.Saludador**).



Compilación y ejecución de un programa Java

7. Errores en el desarrollo de programas

El desarrollo de programas es una actividad compleja y está sujeta a errores, como cualquier otra. Lo primero y más importante que hay que saber de **los errores en el desarrollo de programas es que estos son costosos en tiempo y en recursos**. Un error produce un funcionamiento incorrecto del programa y esto puede generar pérdida de actividad con el consiguiente coste. También requerirá un esfuerzo en la resolución, para lo que habrá que realizar una inversión.

Los errores se producen en cualquiera de las fases del desarrollo de las aplicaciones, por lo que habrá que tomar las medidas de prevención de errores como una actividad transversal que se realizará a lo largo de todo el proyecto.

Durante la fase de la programación, los errores más habituales son los errores sintácticos y de ejecución. Estos errores se detectan durante el proceso de compilación y/o durante el proceso de pruebas, siendo responsabilidad del programador entregar el código fuente libre de errores en la medida de lo posible.

8. Importancia de la reutilización de código

Reutilizar el código consiste en, como su propio nombre indica, utilizar un mismo bloque de código en todas aquellas partes en las que sea necesario. **Reutilizar no significa en ningún caso duplicar,** sino todo lo contrario. Para poder reutilizar el código es necesario agruparlo en unidades funcionales (realizan una acción concreta) que puedan ser utilizadas desde más de un lugar de la aplicación. Estas unidades pueden ser funciones, métodos o clases, dependiendo del paradigma de programación.

La reutilización del código es de **máxima importancia** ya que ahorra costes (tanto para la creación como para la modificación, es mucho más económico actuar en un único lugar) y reduce la posibilidad de cometer errores, ya que es más fácil cometer una equivocación si se debe modificar en varios puntos de la aplicación que en uno único.

Importante

Siempre que haya código duplicado en un programa hay que tratar de evitar esa duplicidad intentando reutilizar el código.

9. Resumen

Los programas informáticos son, de un ordenador, los elementos no físicos (*software*) que convierten a los elementos físicos (*hardware*) en útiles. Gracias a los programas informáticos, un ordenador permite escribir un documento, realizar un cálculo o navegar por la web.

La creación de los programas informáticos es conocida como programación o desarrollo de software. Esta actividad es realizada por un perfil profesional conocido como **programador** que de manera manual y con la ayuda de determinadas herramientas escribe el llamado **código fuente** que será el origen del **programa ejecutable**. Para ello utilizará un **lenguaje de programación**, clasificado a partir de unas características que lo definen.

El desarrollo de aplicaciones debe realizarse de manera ordenada. Para ello, el trabajo se agrupa en fases que son ordenadas de acuerdo a metodologías o ciclos de vida, lo que se conoce como **ingeniería del software.**

El trabajo de creación de los programas informáticos tiene su punto culminante en la obtención del código ejecutable, un proceso conocido generalmente como **compilación**, aunque existen otras formas de ejecución como la **basada en intérpretes** o en **máquinas virtuales**.

Tech Hub <FP> euroformac

Glosario

- **Aplicación informática**: programa o conjunto de programas informáticos que realizan un conjunto de tareas relacionadas entre sí.
- **Análisis:** fase de la construcción de una aplicación informática en la que se determina qué va a hacer la misma.
- **Analista**: perfil profesional que se encarga de realizar el análisis.
- **Bytecode:** código ejecutable destinado a ser ejecutado en una máquina virtual.
- **Ciclo de vida:** en el desarrollo de software, es el conjunto ordenado de fases que hay que llevar a cabo para conseguir completar un proyecto.
- **Código máquina:** también conocido como lenguaje máquina, es el lenguaje comprensible por los ordenadores.
- **Código fuente:** es el programa informático escrito por el programador. No es directamente entendible por la máquina.
- **Código objeto:** es el programa informático capaz de ser entendido por los ordenadores, pero no ejecutable, ya que debe enlazarse con otros programas.
- **Código ejecutable:** es el programa informático capaz de ser entendido y ejecutado por los ordenadores. Se genera como consecuencia del proceso de compilación.
- **Compilación:** proceso de verificación y de transformación del código fuente en código ejecutable.
- **Desarrollo de software:** es la actividad de creación de programas o aplicaciones informáticas.
- **Diseño:** en el desarrollo de software, es la fase en la que se determina cómo se va a realizar el programa o aplicación.
- **Diseñador:** en el desarrollo de software, es la persona encargada de realizar la fase de diseño.
- **Enlazado:** proceso de generación del código ejecutable a partir de los ficheros de código objeto.
- **Intérprete:** programa que verifica y traduce a instrucciones comprensibles por la máquina el código fuente de un programa. La verificación y traducción se realiza durante el propio proceso de ejecución.

- **IDE:** Entorno de Desarrollo Integrado o Integrated Development Environment, es una aplicación informática que integra todas las herramientas necesarias para poder realizar la programación de aplicaciones.
- **Lenguaje de programación:** es un lenguaje formal que contiene las instrucciones y estructuras necesarias para poder hacer programas informáticos.
- **Lenguaje compilado**: es un lenguaje de programación cuyos programas necesitan ser compilados para crear una versión ejecutable.
- **Lenguaje interpretado:** es un lenguaje de programación cuyos programas deben ser traducidos al lenguaje del ordenador durante su ejecución mediante un intérprete.
- **Lenguaje ensamblador:** es un lenguaje de programación de bajo nivel que proporciona un nivel de abstracción superior al del lenguaje máquina.
- **Mantenimiento:** en el ciclo de vida de las aplicaciones informáticas, es el conjunto de actividades de corrección y mejora que se realizan para garantizar el buen funcionamiento de dichas aplicaciones.
- **Máquina virtual:** es un programa que simula ser un ordenador y que hace de puente entre los bytecodes y el ordenador real.
- **Metodología ágil:** son aquellas metodologías que permiten ciclos de trabajo corto y que buscan la reducción de errores y la mejora en el seguimiento de los proyectos.
- **Multiparadigma:** aplicado a los lenguajes informáticos, que admite la programación en varios paradigmas.
- **Paradigma:** aplicado a los lenguajes informáticos, determina la estructura interna de los programas.
- **Plataforma de destino:** sistema operativo y versión al que va destinado un programa ejecutable.
- **Programa ejecutable:** un programa capaz de ser ejecutado directamente en un ordenador.
- **Programador**: perfil profesional cuyo cometido es la programación de aplicaciones.

Bibliografía

- Draper Laboratory. (1969, January 1). Margaret Hamilton - restoration.png (By A. C. Cuerden). Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Margaret_Hamilton_-_restoration.png
- Fernández, Y. (2021, 16 julio). UEFI y BIOS: ¿cuáles son las diferencias? Xataka. https://www.xataka.com/basics/uefi-y-bios-cuales-son-las-diferencias
- Infante, D. C. H. (2023, 6 enero). Los 10 mejores lenguajes de programación para aprender en 2023. Tutoriales Hostinger. https://www.hostinger.es/tutoriales/mejores-lenguajes-de-programacion
- López, J. C. (2020, 8 agosto). John von Neumann, el genio que diseñó la arquitectura de nuestros ordenadores, lo hizo hace 75 años, y este es solo uno de sus logros. Xataka. https://www.xataka.com/historia-tecnologica/john-von-neumann-genio-que-diseno-arquitectura-nuestros-ordenadores-hizo-hace-75-anos-este-solo-uno-sus-logros
- "What is open source?" (2019, 24 octubre). https://www.redhat.com/en/topics/open-source/what-is-open-source